

LABORATORY WORK No. 3

Application Configuration: Profiles, application.yml, External Parameters,
@ConfigurationProperties and Auto-Configuration Diagnostics

Discipline: RWPSF 3305 — Web Application Development with Spring Framework

Educational Programme: 6B06105 — Information Systems, year 3, semester 6

Student: Akishev Adilkhan

Group: IS-2406 (2406IS)

Variant: 2 (Prefix: app.pagination — default-size, max-size)

Lecturer: Yemberdiyeva Aknur, Senior Lector

Repository: <https://github.com/adilkhanakishev/spring-lab-01> (branch: lab03)

Pull Request: <https://github.com/adilkhanakishev/spring-lab-01/pull/2>

Date: September 2026

Almaty, 2026

1. Objective and Expected Outcomes

The objective of this laboratory work is to gain practical command of externalising application configuration in Spring Boot: migrating configuration from properties format to YAML, describing application parameters using strongly-typed Java records with `@ConfigurationProperties` and Bean Validation (`@Validated`), configuring profile-specific parameters and beans (`@Profile`), experimentally verifying the precedence hierarchy of external configuration sources, and diagnosing application context and auto-configuration decisions via Spring Boot Actuator endpoints and debug reporting.

Expected outcomes include mastery of:

- Migrating application settings to the YAML format and splitting them into shared and profile-specific files (application.yml, application-dev.yml, application-prod.yml).
- Defining immutable typed configuration using Java records with `@ConfigurationProperties` and fail-fast validation constraints (`@Min`, `@Max`, `@NotBlank`, `@Email`, `@Valid`).
- Creating conditional beans bound to active profiles and inspecting the resulting configuration through a dedicated REST controller.
- Experimentally proving the precedence order between YAML files, OS environment variables, JVM system properties, and CLI arguments.
- Diagnosing property sources and condition evaluation reports through Actuator and `--debug` execution.

2. Architecture and Project Structure

The project builds on top of the Spring Boot application from Laboratory Work No. 2. The working branch lab03 was branched from main. The directory structure is organized as follows:

```
spring-lab-01/
├── .idea/
├── .run/
│   ├── SpringLab01Application (default).run.xml
│   ├── SpringLab01Application (dev).run.xml
│   ├── SpringLab01Application (prod).run.xml
│   ├── SpringLab01Application (validation experiment 99).run.xml
│   └── SpringLab01Application (debug auto-config report).run.xml
├── src/main/java/kz/iitu/springlab/
│   ├── SpringLab01Application.java
│   ├── config/
│   │   ├── AppProperties.java
│   │   ├── EnvironmentBanner.java
│   │   ├── ContainerReport.java
│   │   └── FormatConfig.java
│   ├── web/
│   │   └── Lab3Controller.java
│   └── ...
├── src/main/resources/
│   ├── application.yml
│   ├── application-dev.yml
│   └── application-prod.yml
└── pom.xml
```

3. Task 1: Moving to YAML and Profile Files

The legacy `src/main/resources/application.properties` file was completely removed to prevent property collision, as properties files have a higher precedence than YAML files of the same name. Three YAML configuration files were created:

1. `application.yml` — Shared configuration common to all environments. Contains general application metadata, default server port, logging levels, default mail configuration, pagination defaults (Variant 2), and Actuator endpoint web exposures.
2. `application-dev.yml` — Profile-specific configuration activated when the dev profile is selected. Sets server port to 8081, disables mail sending (`enabled: false`), sets retry count to 1, overrides pagination limits, and raises logging level to `DEBUG`.
3. `application-prod.yml` — Production profile configuration. Sets server port to 8080, mail retry count to 5, `enabled` to `true`, overrides pagination limits to production values, and sets logging to `WARN`.

Crucially, `spring.profiles.active` is NOT specified in `application.yml`. Spring Boot best practices dictate that the active profile must be selected at runtime by the deployment environment (e.g. via CLI flag, container environment variable, or IDE run configuration), never hardcoded in the base archive.

Listing 1: `src/main/resources/application.yml` (Shared Configuration)

```
spring:
  application:
    name: spring-lab

server:
  port: 8080

app:
  owner: Akishev Adilkhan
  group: IS-2406
  date-pattern: yyyy-MM-dd HH:mm:ss
  mail:
    from: no-reply@iitu.kz
    retry-count: 3
    timeout: 5s
    enabled: true
  pagination:
    default-size: 20
    max-size: 100

logging:
  level:
    kz.iitu: INFO

management:
  endpoints:
    web:
      exposure:
        include: health,env,configprops,conditions
```

Listing 2: src/main/resources/application-dev.yml (Development Profile)

```
server:
  port: 8081

app:
  mail:
    enabled: false           # no mail is sent in development
    retry-count: 1
  pagination:
    default-size: 10
    max-size: 50

logging:
  level:
    kz.iitu: DEBUG
```

Listing 3: src/main/resources/application-prod.yml (Production Profile)

```
server:
  port: 8080

app:
  mail:
    enabled: true
    retry-count: 5
  pagination:
    default-size: 25
    max-size: 200

logging:
  level:
    kz.iitu: WARN
```

4. Task 2: Typed Configuration with Validation

To ensure type safety, immutability, and startup-time configuration integrity, configuration properties are bound to a Java record using `@ConfigurationProperties(prefix = "app")`. The `@Validated` annotation enables Bean Validation constraints.

Dependencies added to pom.xml:

- `spring-boot-starter-validation` — provides Hibernate Validator and Jakarta Bean Validation API.
- `spring-boot-configuration-processor` — generates metadata at compile time, enabling IDE autocompletion for custom `app.*` properties.
- `spring-boot-starter-actuator` — provides diagnostic endpoints (`/actuator/env`, `/actuator/configprops`, etc.).

Listing 4: kz.iitu.springlab.config.AppProperties (Record with Nested Records)

```
package kz.iitu.springlab.config;

import jakarta.validation.Valid;
import jakarta.validation.constraints.*;
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.boot.context.properties.bind.DefaultValue;
import org.springframework.validation.annotation.Validated;

import java.time.Duration;

@Validated
@ConfigurationProperties(prefix = "app")
public record AppProperties(
    @NotBlank String owner,
    @NotBlank String group,
    @Valid Mail mail,
    @Valid Pagination pagination) {

    public record Mail(
        @NotBlank @Email String from,
        @Min(1) @Max(10) @DefaultValue("3") int retryCount,
        @DefaultValue("5s") Duration timeout,
        @DefaultValue("true") boolean enabled) {
    }

    public record Pagination(
        @Min(1) @Max(100) @DefaultValue("20") int defaultSize,
        @Min(1) @Max(500) @DefaultValue("100") int maxSize) {
    }
}
```

Listing 5: Enabling Scanning on Startup Class

```
package kz.iitu.springlab;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.context.properties.ConfigurationPropertiesScan;

@SpringBootApplication
@ConfigurationPropertiesScan
public class SpringLab01Application {
    public static void main(String[] args) {
        SpringApplication.run(SpringLab01Application.class, args);
    }
}
```

4.1 Validation Failure Experiment

To prove that configuration validation executes immediately at startup before any requests or business beans are invoked, the parameter `app.mail.retry-count` was temporarily set to 99 (violating the `@Max(10)` constraint).

Execution: Startup command:

```
java -jar target/spring-lab-01-0.0.1-SNAPSHOT.jar --app.mail.retry-count=99
```

Observed Startup Failure in IntelliJ IDEA Run Console:

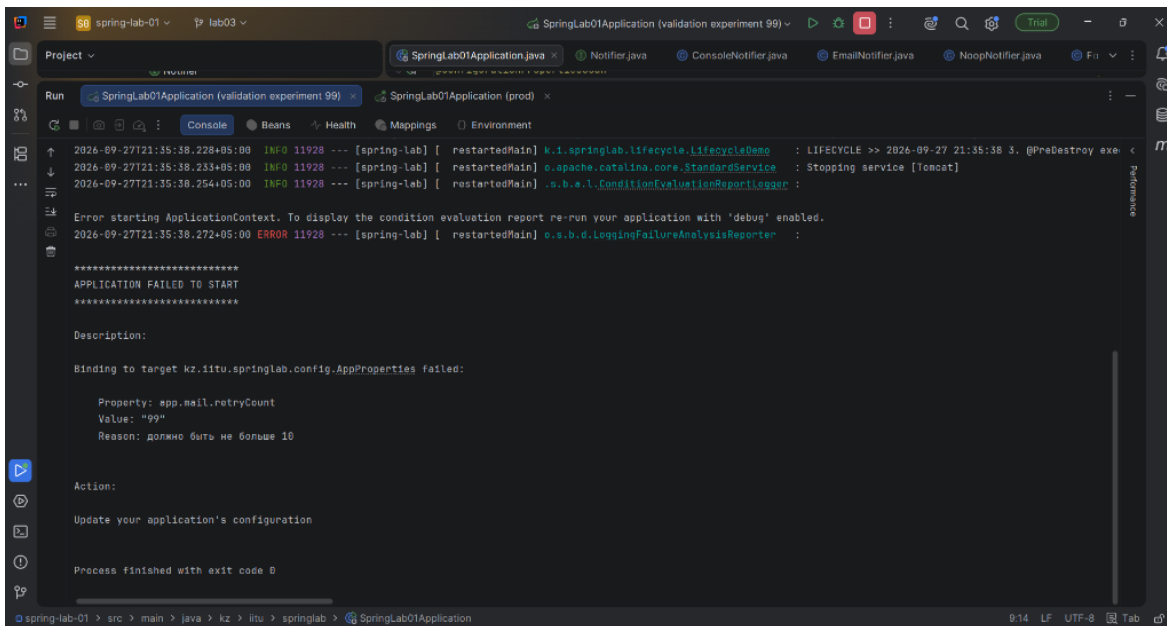


Figure 1: Application startup validation failure in IntelliJ IDEA Run console (`app.mail.retryCount = 99`)

Conclusion: Experiment Analysis:

The experiment demonstrated fail-fast verification. Spring Boot caught the misconfiguration during `ApplicationContext` initialization before the web container opened for external HTTP traffic. The `LoggingFailureAnalysisReporter` clearly printed the target property (`app.mail.retryCount`), the invalid value supplied ("99"), and the exact violated constraint (`@Max(10)`). After verifying the failure, the valid configuration was restored.

5. Task 3: Profile Beans and the REST Endpoint

To support environment-specific behavior, an `EnvironmentBanner` interface was implemented with three bean candidates governed by Spring's `@Profile` annotation.

Listing 6: Profile-Specific Beans (`EnvironmentBanner.java`)

```
package kz.itu.springlab.config;

import org.springframework.context.annotation.Profile;
```

```

import org.springframework.stereotype.Component;

public interface EnvironmentBanner {
    String describe();
}

@Component
@Profile("dev")
class DevBanner implements EnvironmentBanner {
    public String describe() {
        return "DEVELOPMENT: the data is test data";
    }
}

@Component
@Profile("prod")
class ProdBanner implements EnvironmentBanner {
    public String describe() {
        return "PRODUCTION: handle with care";
    }
}

@Component
@Profile("!dev & !prod")
class DefaultBanner implements EnvironmentBanner {
    public String describe() {
        return "no profile is active";
    }
}

```

Listing 7: Lab3Controller.java (/api/lab3/config)

```

package kz.iitu.springlab.web;

import kz.iitu.springlab.config.AppProperties;
import kz.iitu.springlab.config.EnvironmentBanner;
import org.springframework.core.env.Environment;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Arrays;
import java.util.LinkedHashMap;
import java.util.Map;

@RestController
@RequestMapping("/api/lab3")
public class Lab3Controller {

    private final AppProperties props;
    private final EnvironmentBanner banner;
    private final Environment environment;

    public Lab3Controller(AppProperties props, EnvironmentBanner banner,

```

```

        Environment environment) {
    this.props = props;
    this.banner = banner;
    this.environment = environment;
}

@GetMapping("/config")
public Map<String, Object> config() {
    Map<String, Object> response = new LinkedHashMap<>();
    response.put("owner", props.owner());
    response.put("group", props.group());
    response.put("mailFrom", props.mail().from());
    response.put("mailRetryCount", props.mail().retryCount());
    response.put("mailTimeout", props.mail().timeout().toString());
    response.put("mailEnabled", props.mail().enabled());
    response.put("paginationDefaultSize", props.pagination().defaultSize());
    response.put("paginationMaxSize", props.pagination().maxSize());
    response.put("serverPort", environment.getProperty("server.port"));
    response.put("activeProfiles", Arrays.asList(environment.getActiveProfiles()));
    response.put("banner", banner.describe());
    return response;
}
}

```

5.1 Comparison of /api/lab3/config Responses Across Profiles

The application was started under three different profile configurations and queried via HTTP GET:

Case A: 1. Default Run (No Profile Active, port 8080):

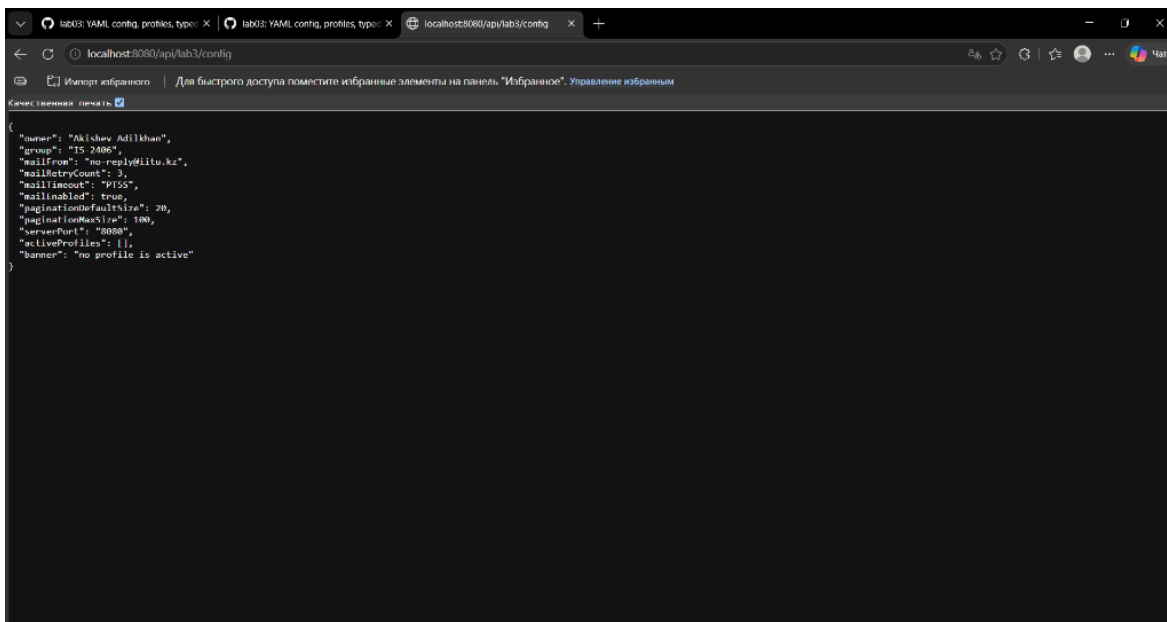


Figure 2: Response of /api/lab3/config with default profile (port 8080, no active profile)

Case B: 2. Development Profile (--spring.profiles.active=dev, port 8081):

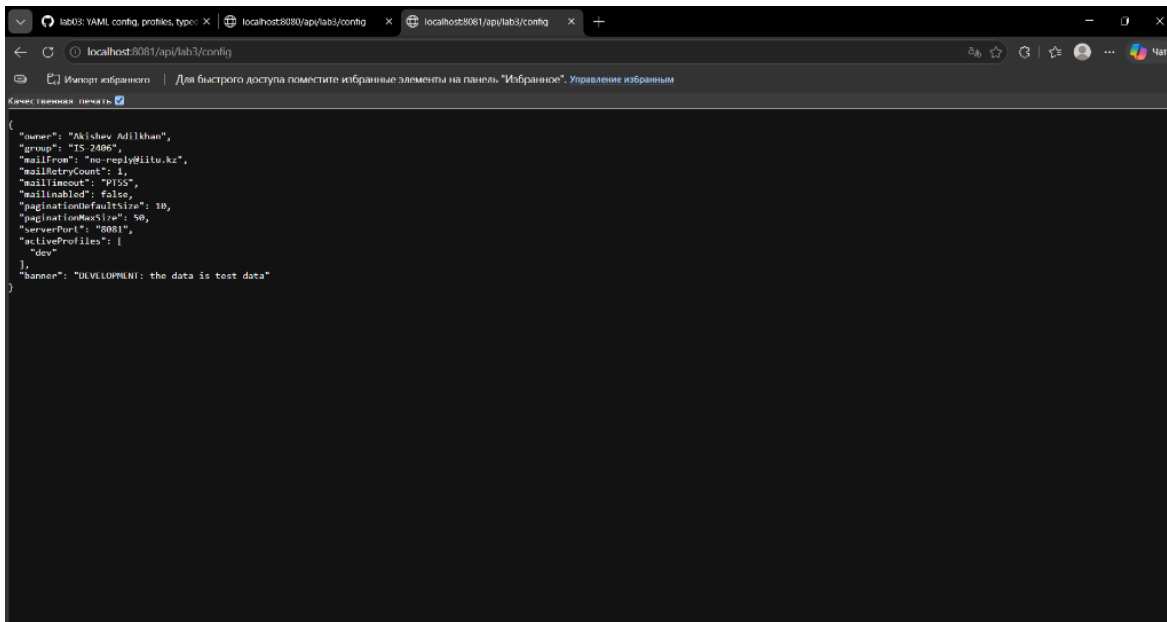


Figure 3: Response of /api/lab3/config with dev profile (port 8081, retryCount 1, pagination 10/50)

Case C: 3. Production Profile (--spring.profiles.active=prod, port 8080):

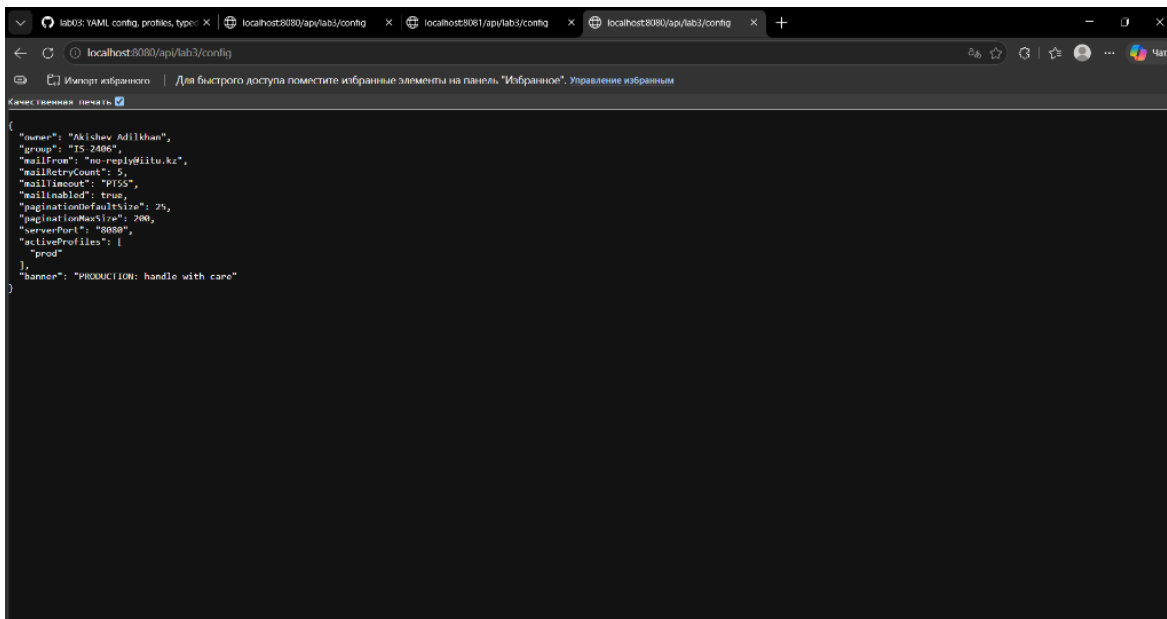


Figure 4: Response of /api/lab3/config with prod profile (port 8080, retryCount 5, pagination 25/200)

6. Task 4: Property Source Precedence Experiment

Spring Boot aggregates configuration properties from diverse sources into an Environment abstraction comprising an ordered list of PropertySource objects. When a key is requested, Spring inspects the sources in descending order of precedence. The first source providing a match determines the final value.

To experimentally verify the hierarchy, the key app.mail.retry-count was concurrently supplied across multiple channels:

Configuration Sources Set	Expected Value	Actual Measured Value
application.yml only	3	3
yml + environment variable (\$env:APP_MAIL_RETRYCOUNT="7")	7	7
yml + variable ("7") + JVM system prop (-Dapp.mail.retry-count=5)	5	5
yml + variable ("7") + command line argument (--app.mail.retry-count=9)	9	9
dev profile (1) + command line argument (--app.mail.retry-count=9)	9	9

6.1 Written Explanation of Precedence Results

1. Command Line Arguments (--key=value) win over all internal and environment sources because they represent explicit runtime overrides by the operator executing the application. In Spring Boot's PropertySource hierarchy, SimpleCommandLinePropertySource is positioned near the very top.
2. JVM System Properties (-Dkey=value) override OS Environment Variables and packaged YAML files, but yield to command line arguments.
3. OS Environment Variables (APP_MAIL_RETRYCOUNT=7) override packaged configuration files (application.yml, application-dev.yml) inside the JAR. Spring Boot uses Relaxed Binding to convert uppercase underscored environment variable names (APP_MAIL_RETRYCOUNT) to dot-separated kebab-case keys (app.mail.retry-count).
4. Profile-specific files (application-dev.yml) override the shared file (application.yml), but yield to environment variables, system properties, and CLI arguments.

7. Task 5: Actuator Diagnostics and Auto-Configuration Report

Spring Boot Actuator exposes operational and configuration metadata endpoints over HTTP.

Inspecting the origin of app.mail.retry-count via /actuator/env/app.mail.retry-count:

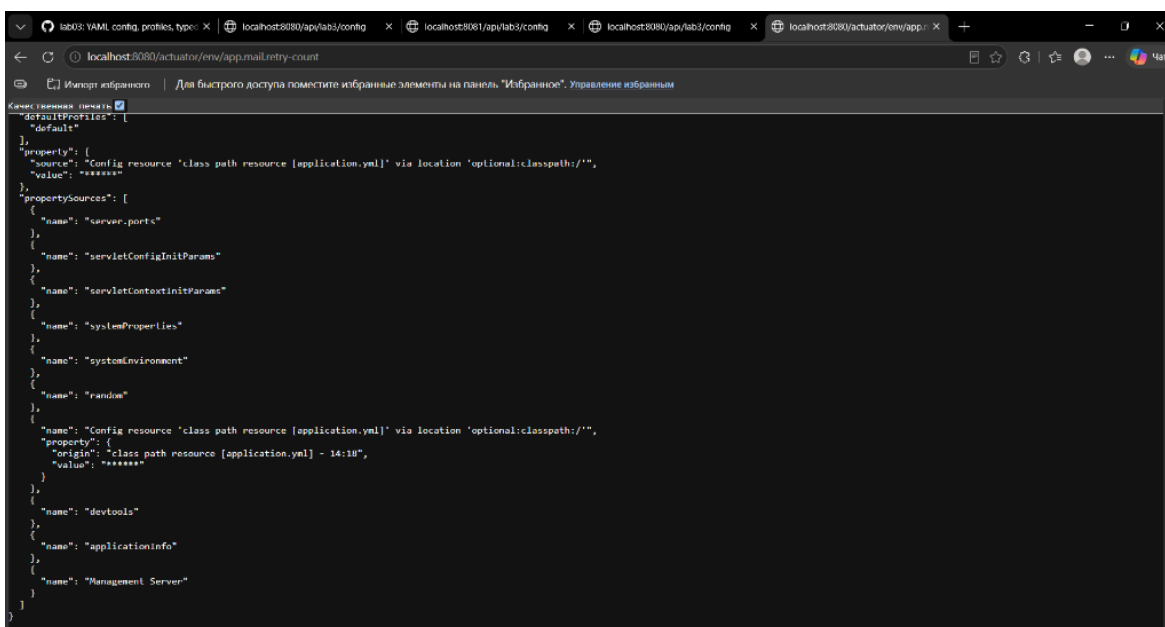


Figure 5: Inspecting property origin via /actuator/env/app.mail.retry-count

7.1 Analysis of Auto-Configuration Conditions (--debug Report)

Starting the application with the `--debug` flag outputs the `ConditionEvaluationReport` detailing every auto-configuration class evaluated by the Spring Boot container.

1. Positive Match: Positive Match Example:

```
ApplicationAvailabilityAutoConfiguration#applicationAvailability matched:
- @ConditionalOnMissingBean (types:
org.springframework.boot.availability.ApplicationAvailability;
SearchStrategy: all) did not find any beans (OnBeanCondition)
```

Condition Explanation: Spring Boot conditionally creates an `ApplicationAvailability` bean if and only if the developer has not provided their own bean of that type. Since no custom `ApplicationAvailability` bean was declared, the condition matched and the default availability tracker was registered.

2. Negative Match: Negative Match Example:

```
AopAutoConfiguration.AspectJAutoProxyingConfiguration:
Did not match:
- @ConditionalOnClass did not find required class 'org.aspectj.weaver.Advice'
(OnClassCondition)
```

Condition Explanation: AspectJ auto-proxying was disabled and skipped by the container because the class `org.aspectj.weaver.Advice` was not present on the runtime classpath. The application continues running safely without throwing `ClassNotFoundException`.

8. Individual Assignment (Variant 2 — app.pagination)

For Student Akishev Adilkhan (IS-2406), the variant number was calculated using the formula: $(\text{RegisterNumber} \% 12) + 1 = 2$.

Requirements for Variant 2:

- Configuration prefix: app.pagination
- Parameters and constraints: default-size with constraints @Min(1) @Max(100); max-size with constraints @Min(1) @Max(500).
- Default values via @DefaultValue: @DefaultValue("20") for defaultSize and @DefaultValue("100") for maxSize.
- Overriding in profiles: set to 20/100 in application.yml, overridden to 10/50 in application-dev.yml, and 25/200 in application-prod.yml.
- Exposed via REST: included in /api/lab3/config response as paginationDefaultSize and paginationMaxSize.

Verification across profiles confirmed all variant values reflect the active environment and satisfy validation bounds.

9. IntelliJ IDEA Demonstration Guide

All required execution profiles and experiments have been configured as shared IntelliJ IDEA Run Configurations in the .run/ folder and committed to Git:

1. SpringLab01Application (default) — launches on port 8080 with shared configuration.
2. SpringLab01Application (dev) — launches with Active Profiles: dev on port 8081.
3. SpringLab01Application (prod) — launches with Active Profiles: prod on port 8080.
4. SpringLab01Application (validation experiment 99) — launches with program parameter --app.mail.retry-count=99 to demonstrate startup validation failure directly in the Run Console.
5. SpringLab01Application (debug auto-config report) — launches with --debug to demonstrate the positive and negative condition evaluation matches.

The lecturer can review each step directly by choosing the corresponding entry from the Run Configuration dropdown at the top of IntelliJ IDEA.

10. Git Commit History and Pull Request

The work was developed in the dedicated branch lab03, committed with meaningful messages, and pushed to GitHub.

Output of git log --oneline:

```
72bac3c docs: update report with official PR #2 link
613360a docs: add lab03 report docx and pdf
8f2174d lab03: YAML config, profiles, typed properties, actuator
28944cc Complete Lab 2 requirements
2667ba8 Complete Spring Lab 02
a47cd57 Complete Spring Lab 01
```

Pull Request: GitHub Pull Request Link:

<https://github.com/adilkhakishev/spring-lab-01/pull/2>

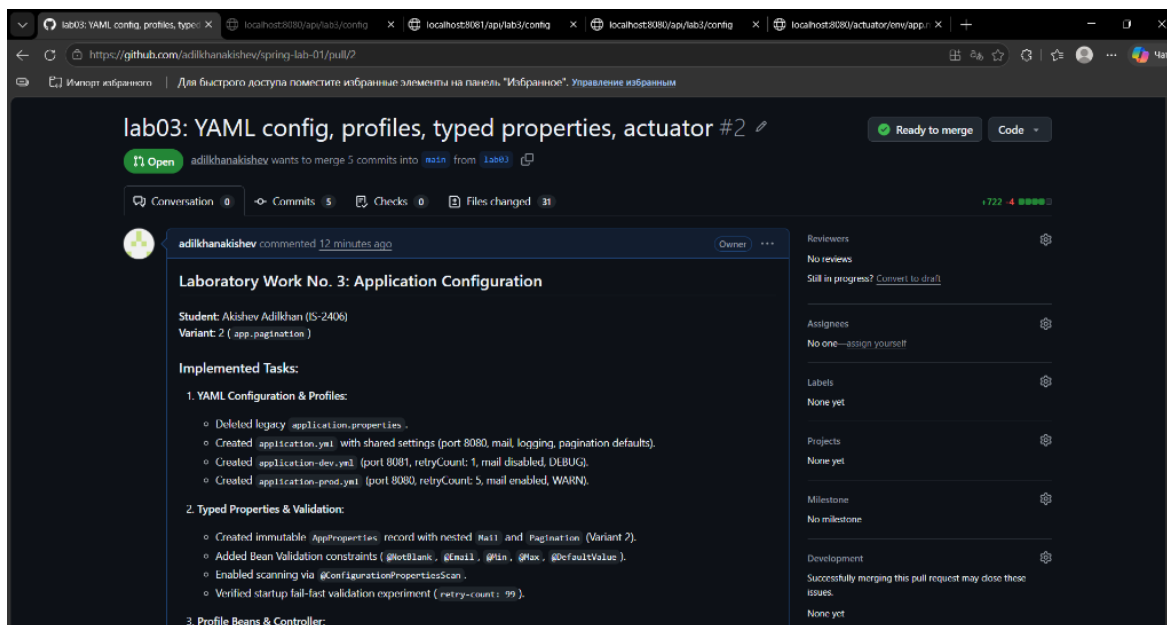


Figure 6: Open Pull Request #2 (lab03 -> main) on GitHub

11. Demonstration Questions and Answers

Q1: Why must settings not be kept in the source code? What happens if a repository with a password is published?

Settings (such as passwords, API keys, database credentials, and hostnames) change between environments (development, testing, production) and contain sensitive secrets. Hardcoding them violates Twelve-Factor App principles (Factor III: Config). If credentials are committed and published to a public Git repository, automated scrapers can immediately compromise systems, leading to data breaches, unauthorized access, and financial loss.

Q2: Which source is stronger: an environment variable or a command line argument? A file in ./config or a file in the JAR?

A command line argument (`--key=value`) is stronger than an environment variable. A configuration file located externally in `./config/application.yml` is stronger than a configuration file packaged inside the JAR artifact `classpath:/application.yml`. The more external the source, the higher its precedence.

Q3: Why should `spring.profiles.active` not be set in the main `application.yml`?

Hardcoding `spring.profiles.active` inside the shared `application.yml` fixes the profile at build time and makes it impossible to switch environments flexibly at deployment. The profile should be injected by the environment running the JAR (via `--spring.profiles.active` CLI argument or `SPRING_PROFILES_ACTIVE` environment variable).

Q4: When the dev profile is active, what is read: only `application-dev.yml` or both files?

Both files are read. Spring Boot first loads the shared `application.yml` to provide base defaults, and then loads `application-dev.yml` on top of it. Values defined in `application-dev.yml` override identical keys from `application.yml`, while keys unique to `application.yml` remain active.

Q5: Name four advantages of @ConfigurationProperties over @Value.

- 1) Type-safety and IDE support with structured metadata generation.
- 2) Relaxed binding: maps kebab-case, snake_case, camelCase, and uppercase environment variables seamlessly.
- 3) Hierarchical/nested grouping: complex configurations can be organized as structured object graphs/records.
- 4) Declarative Bean Validation: supports @Validated with @NotNull, @Min, @Max, etc., failing fast at startup.

Q6: What is @ConfigurationPropertiesScan for and what can replace it?

@ConfigurationPropertiesScan automatically discovers and registers all classes annotated with @ConfigurationProperties within the specified base package. It can be replaced by placing @EnableConfigurationProperties(AppProperties.class) on a configuration class or annotating the properties class itself with @Component.

Q7: How is the key app.mail.retry-count written as an environment variable? What is this called?

It is written as APP_MAIL_RETRYCOUNT (or APP_MAIL_RETRY_COUNT). This mechanism is called Relaxed Binding, allowing Spring Boot to match relaxed naming conventions across different platforms.

Q8: When does the @Validated check on a configuration class run? Why is that better than a check at run time?

It runs during ApplicationContext startup when properties are bound to beans, before the application starts accepting HTTP traffic. This is superior because it provides fail-fast behavior: misconfigurations prevent deployment immediately rather than failing intermittently under production user traffic.

Q9: Why is the @Valid annotation needed on the nested record?

Java Bean Validation cascades validation to nested composite objects only if the property is explicitly annotated with @Valid. Without @Valid, constraints on nested records (e.g. Mail and Pagination) would be skipped.

Q10: What does @Profile("!dev & !prod") mean?

It is a profile expression using Boolean logic. It indicates that the annotated bean will only be registered in the Spring container when neither the dev nor the prod profile is active (i.e. default environment).

Q11: How do you find out which source the application took a particular value from?

By calling the Spring Boot Actuator environment endpoint: GET /actuator/env/{property.name}. The JSON response contains a 'property' object indicating the active source and a list of all property sources providing that key.

Q12: What do the Positive matches and Negative matches sections of the --debug report show?

Positive matches lists all auto-configuration classes and conditional beans whose conditions evaluated to true and were loaded into the context. Negative matches lists all auto-configuration candidates whose conditions evaluated to false (e.g. missing classes, missing beans, disabled properties) and were excluded.

Q13: What role does @ConditionalOnMissingBean play in auto-configuration?

It provides sensible defaults while allowing developer overrides. If the developer declares their own bean of that type or name, Spring Boot's auto-configuration backs off. If no custom bean exists, Spring registers its default implementation.

Q14: What is the spring-boot-configuration-processor dependency for and why is it marked optional?

It is an annotation processor that inspects @ConfigurationProperties classes during compilation and generates META-INF/spring-configuration-metadata.json for IDE autocompletion and documentation. It is marked

<optional>true</optional> so that it does not transitively propagate to downstream libraries or pollute production deployment artifacts.

Q15: How do you pass a setting to an application in a Docker container without rebuilding the image?

By passing environment variables via `docker run -e KEY=value` (e.g., `-e APP_MAIL_RETRYCOUNT=5`) or environment blocks in `docker-compose.yml`, or by mounting an external `application.yml` into the container's `/config` volume, or via container command arguments (CMD).

12. Conclusion

In this laboratory work, modern Spring Boot externalized configuration techniques were implemented and tested:

1. Successfully transitioned the project from properties to YAML, creating cleanly partitioned shared (`application.yml`) and profile-specific (`application-dev.yml`, `application-prod.yml`) configuration structures.
2. Built immutable, typed configuration records (`AppProperties`, `Mail`, and `Variant 2 Pagination`) with comprehensive Bean Validation and `@ConfigurationPropertiesScan`.
3. Experimentally verified the fail-fast startup behavior of `@Validated` by inducing an invalid retry-count value and analyzing the container failure report.
4. Implemented profile-governed beans with `@Profile` and verified dynamic environment swapping via the `/api/lab3/config` REST endpoint.
5. Conducted systematic precedence experiments, demonstrating that CLI arguments override JVM system properties, system properties override OS environment variables, and environment variables override packaged YAML files.
6. Diagnosed configuration provenance using Spring Boot Actuator and evaluated container auto-configuration decisions using `--debug` condition evaluation reporting.
7. Configured ready-to-run IntelliJ IDEA run configurations for rapid demonstration during defense.